

Chapter 7. Cluster Analysis

Dong-Kyu Chae

**PI of the Data Intelligence Lab @HYU
Department of Computer Science & Data Science
Hanyang University**



DBSCAN Algorithm

ALGORITHM 1: Pseudocode of Original Sequential DBSCAN Algorithm

```
Input: DB: Database
Input:  $\epsilon$ : Radius
Input: minPts: Density threshold
Input: dist: Distance function
Data: label: Point labels, initially undefined

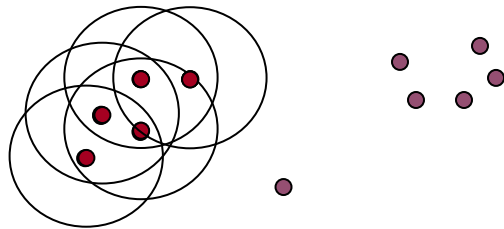
1 foreach point p in database DB do                                // Iterate over every point
2   if label(p)  $\neq$  undefined then continue                        // Skip processed points
3   Neighbors N  $\leftarrow$  RANGEQUERY(DB, dist, p,  $\epsilon$ )           // Find initial neighbors
4   if  $|N| < \textit{minPts}$  then                                          // Non-core points are noise
5     label(p)  $\leftarrow$  Noise
6     continue
7   c  $\leftarrow$  next cluster label                                    // Start a new cluster
8   label(p)  $\leftarrow$  c
9   Seed set S  $\leftarrow N \setminus \{p\}$                             // Expand neighborhood
10  foreach q in S do
11    if label(q) = Noise then label(q)  $\leftarrow$  c
12    if label(q)  $\neq$  undefined then continue
13    Neighbors N  $\leftarrow$  RANGEQUERY(DB, dist, q,  $\epsilon$ )
14    label(q)  $\leftarrow$  c
15    if  $|N| < \textit{minPts}$  then continue                                // Core-point check
16    S  $\leftarrow S \cup N$ 
```



DBSCAN Algorithm: Example

□ Parameter

- $\varepsilon = 2 \text{ cm}$
- $MinPts = 3$



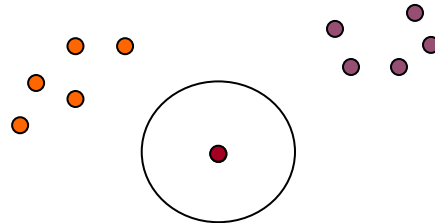
```
for each  $o \in D$  do  
  if  $o$  is not yet marked then  
    if  $o$  is a core-object then  
      collect all objects density-reachable from  $o$   
      and assign them to a new cluster.  
    else  
      assign  $o$  to NOISE
```



DBSCAN Algorithm: Example

□ Parameter

- $\varepsilon = 2 \text{ cm}$
- $MinPts = 3$



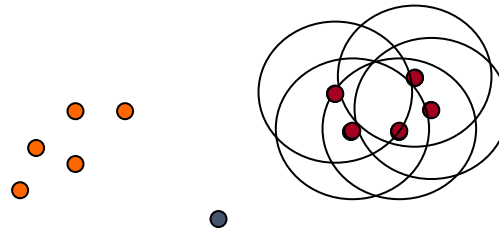
```
for each  $o \in D$  do  
  if  $o$  is not yet marked then  
    if  $o$  is a core-object then  
      collect all objects density-reachable from  $o$   
      and assign them to a new cluster.  
    else  
      assign  $o$  to NOISE
```



DBSCAN Algorithm: Example

□ Parameter

- $\varepsilon = 2 \text{ cm}$
- $MinPts = 3$



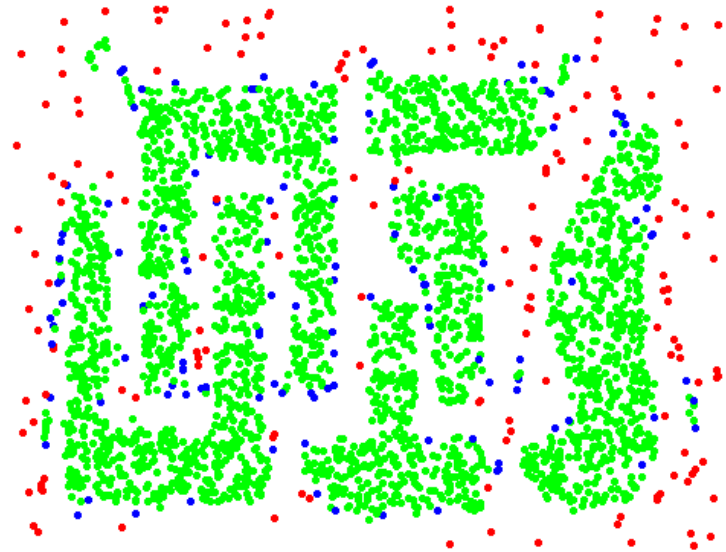
```
for each  $o \in D$  do  
  if  $o$  is not yet marked then  
    if  $o$  is a core-object then  
      collect all objects density-reachable from  $o$   
      and assign them to a new cluster.  
    else  
      assign  $o$  to NOISE
```



Point Types Marked by DBSCAN



Original Points



Point types: **core**,
border and **outliers**

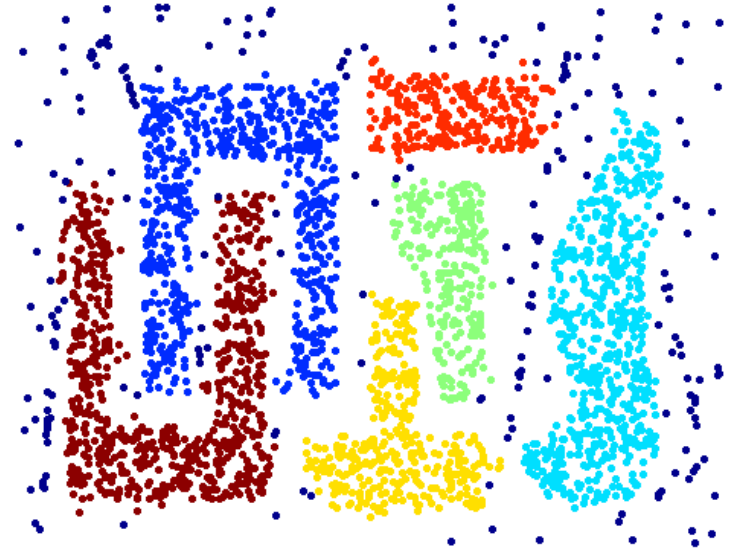
$\epsilon = 10$, MinPts = 4



When DBSCAN Works Well



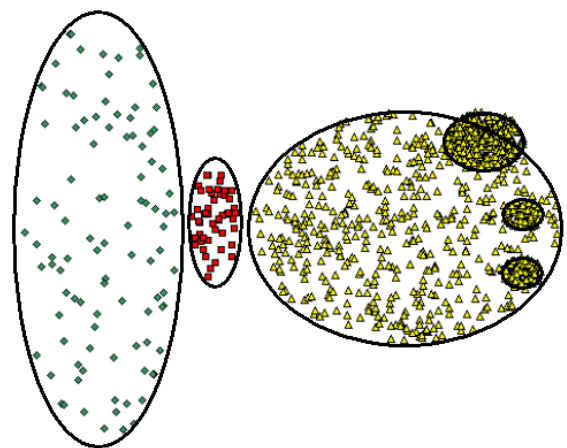
Original Points



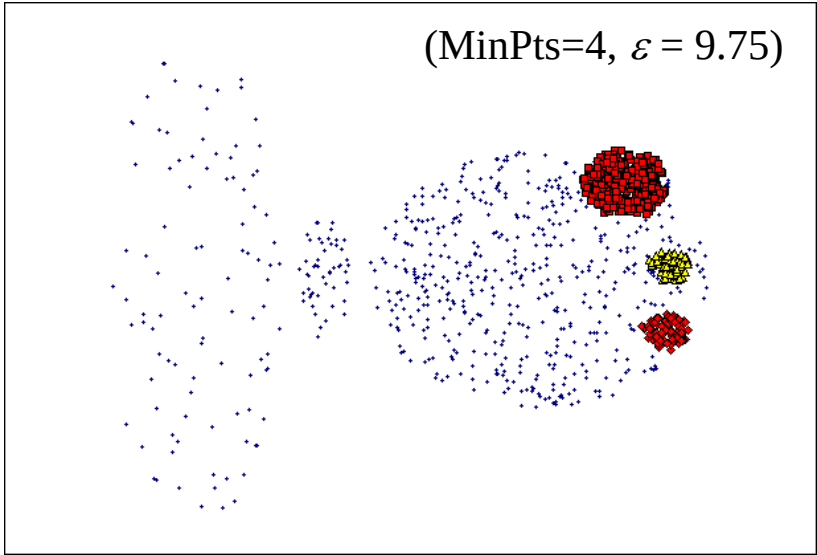
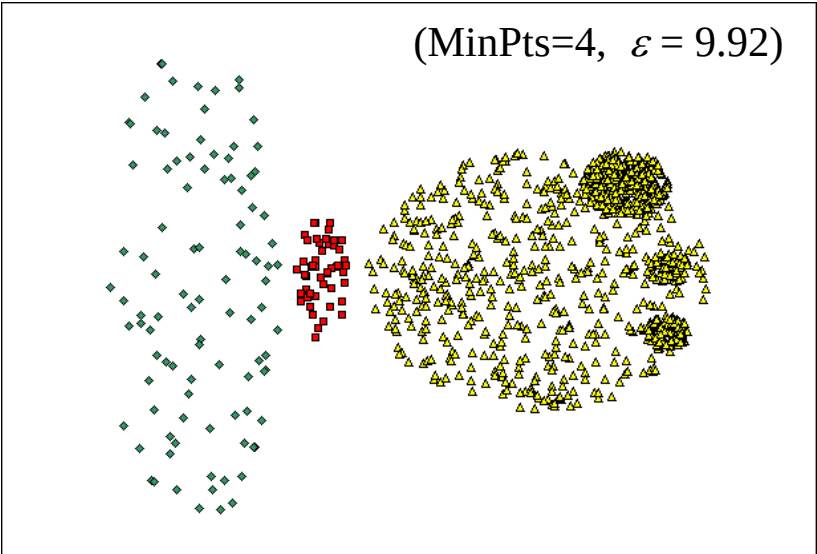
Clusters

- **Resistant to Noise**
- **Can handle clusters of different shapes and sizes**

DBSCAN: Sensitive to Parameters



Original Points



DBSCAN: Sensitive to Parameters

Figure 8. DBScan results for DS1 with MinPts at 4 and Eps at (a) 0.5 and (b) 0.4.

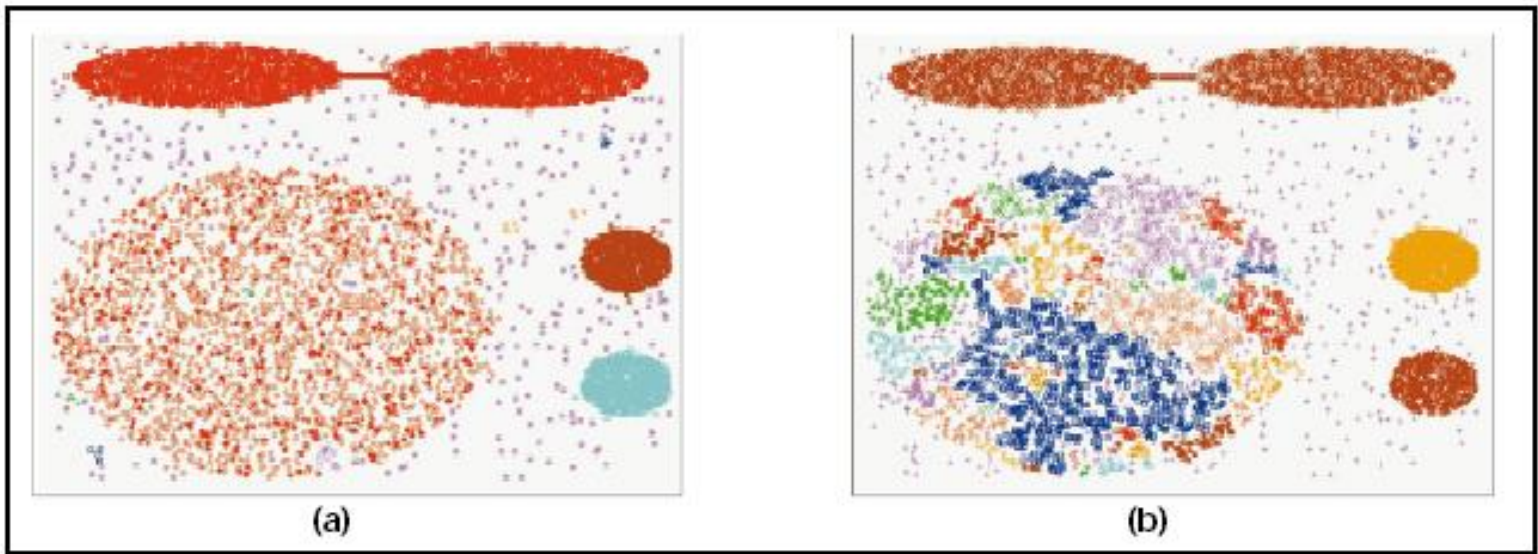
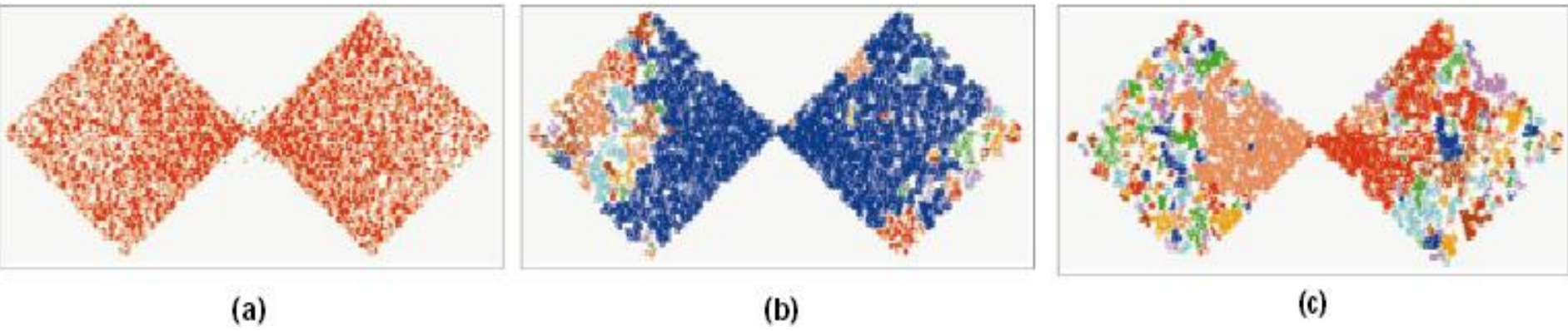


Figure 9. DBScan results for DS2 with MinPts at 4 and Eps at (a) 5.0, (b) 3.5, and (c) 3.0.



DBSCAN Summary

❑ Advantages

- ❑ Clusters can have arbitrary shape and size
- ❑ Number of clusters is determined automatically
- ❑ Can separate clusters from noise and outliers

❑ Disadvantages

- ❑ Input parameters may be difficult to determine
- ❑ In some situations very sensitive to input parameter setting

❑ OPTICS

- ❑ Based on DBSCAN
- ❑ Does not produce clusters explicitly
- ❑ Rather generate **an ordering of data objects** representing density-based clustering structure



OPTICS: Some Extension from DBSCAN

- ❑ **OPTICS: Ordering Points To Identify the Clustering Structure**
 - ❑ It aims to answer: *"how to choose proper ε value?"*
 - ❑ Produces a linear **order** of objects such that **spatially closest points become neighbors in the ordering**
 - ❑ This ordering can produce a **graphical information** equivalent to density-based clustering structure corresponding to a broad range of parameter settings (ε)
 - ❑ OPTICS can be seen as a **visualization technique** for clustering, rather than a clustering solution
 - ❑ Good for both automatic and interactive cluster analysis, including finding intrinsic clustering structure

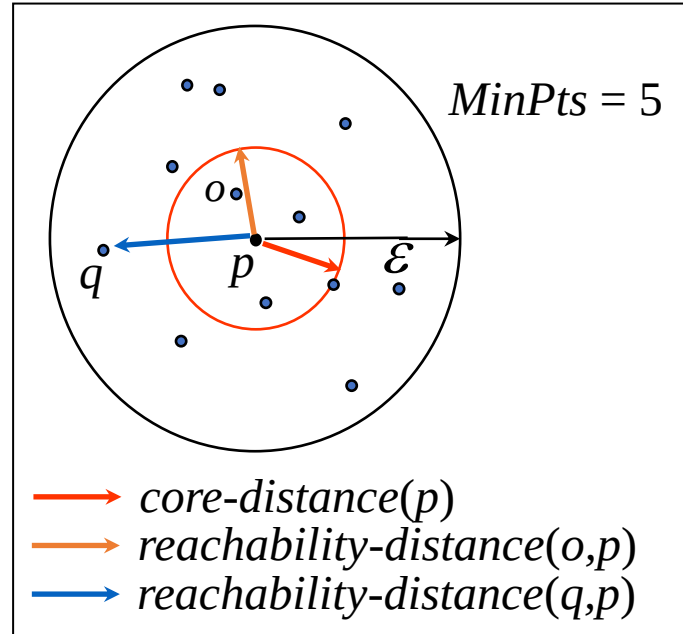
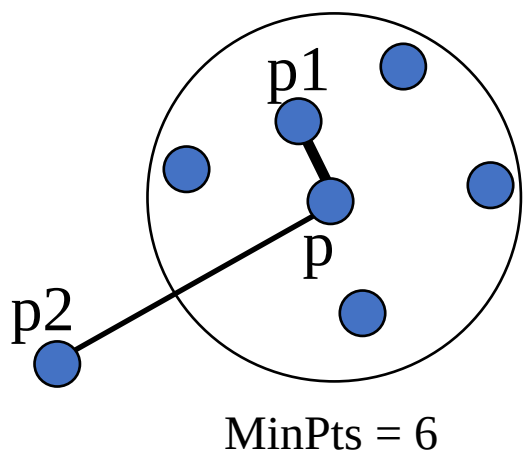
OPTICS: Some Extension from DBSCAN

$\square \text{ core-dist}_{\epsilon, MinPts}(p) = \begin{cases} \text{UNDEFINED} & \text{if } |N_{\epsilon}(p)| < MinPts \\ MinPts\text{-th smallest distance in } N_{\epsilon}(p) & \text{otherwise} \end{cases}$

“smallest distance that makes *p* a **core**”

$\square \text{ reachability-dist}_{\epsilon, MinPts}(o, p) = \begin{cases} \text{UNDEFINED} & \text{if } |N_{\epsilon}(p)| < MinPts \\ \max(\text{core-dist}_{\epsilon, MinPts}(p), \text{dist}(p, o)) & \text{otherwise} \end{cases}$

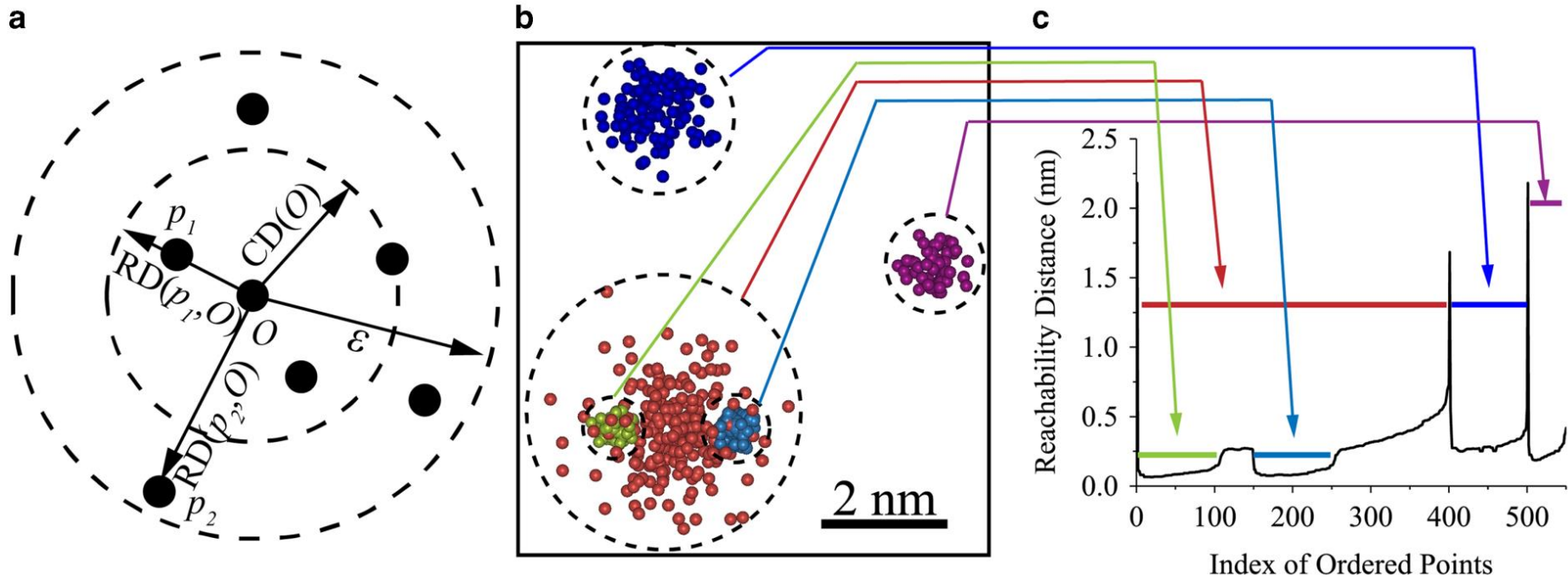
“smallest distance that makes *o* **directly density-reachable** from *p*”



OPTICS: Some Extension from DBSCAN

□ Idea

- Order data points by ***shortest reachability distance***
- The clusters show up as **valleys** in the **reachability plot**. The deeper the valley, the denser the cluster (having shorter reachability distances).
- Extracting clusters from this plot can be done manually by selecting a threshold on the y-axis
- Needs an initial ϵ^* , but it does not require a careful decision.

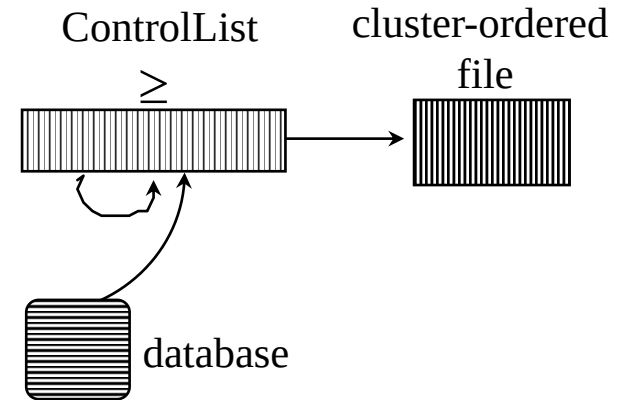


The Pseudocode of OPTICS (skip)

□ Output: ordered data points

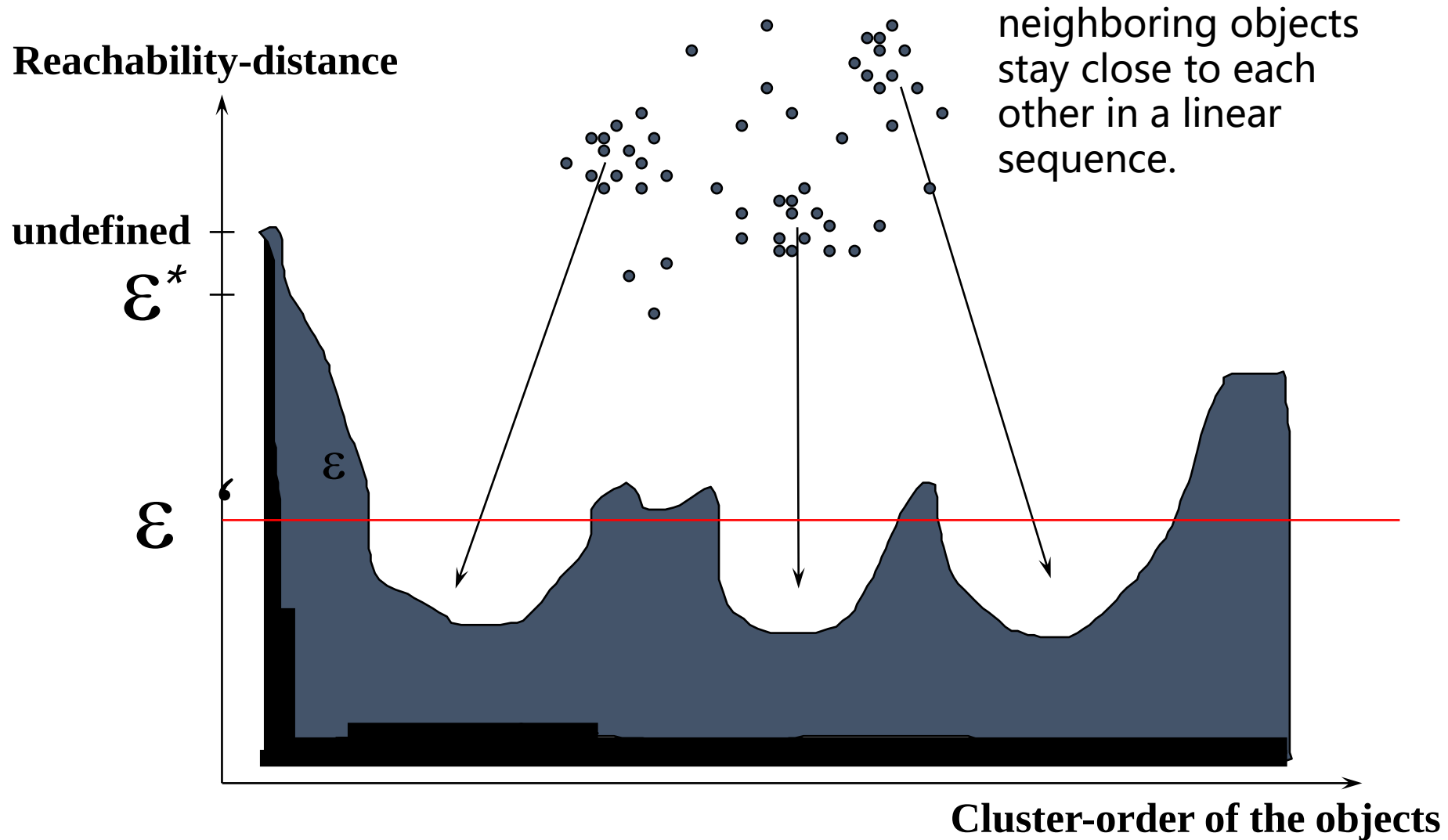
```

foreach  $o \in \text{Database}$ 
  // initially,  $o.\text{processed} = \text{false}$  for all objects  $o$ 
  if  $o.\text{processed} = \text{false}$ ;
    insert ( $o$ , “undefined”) into ControlList;
  while ControlList is not empty
    select first element ( $o$ ,  $r\_dist$ ) from ControlList;
    retrieve  $N_\epsilon(o)$  and determine  $c\_dist = \text{core-distance}(o)$ ;
    set  $o.\text{processed} = \text{true}$ ;
    write ( $o$ ,  $r\_dist$ ,  $c\_dist$ ) to file;
    if  $o$  is a core object at any distance  $\leq \epsilon$ 
      foreach  $p \in N_\epsilon(o)$  not yet processed;
        determine  $r\_dist_p = \text{reachability-distance}(p, o)$ ;
        if  $(p, \_) \notin \text{ControlList}$ 
          insert ( $p$ ,  $r\_dist_p$ ) in ControlList;
        else if  $(p, \text{old\_r\_dist}) \in \text{ControlList}$  and  $r\_dist_p < \text{old\_r\_dist}$ 
          update ( $p$ ,  $r\_dist_p$ ) in ControlList;
  
```

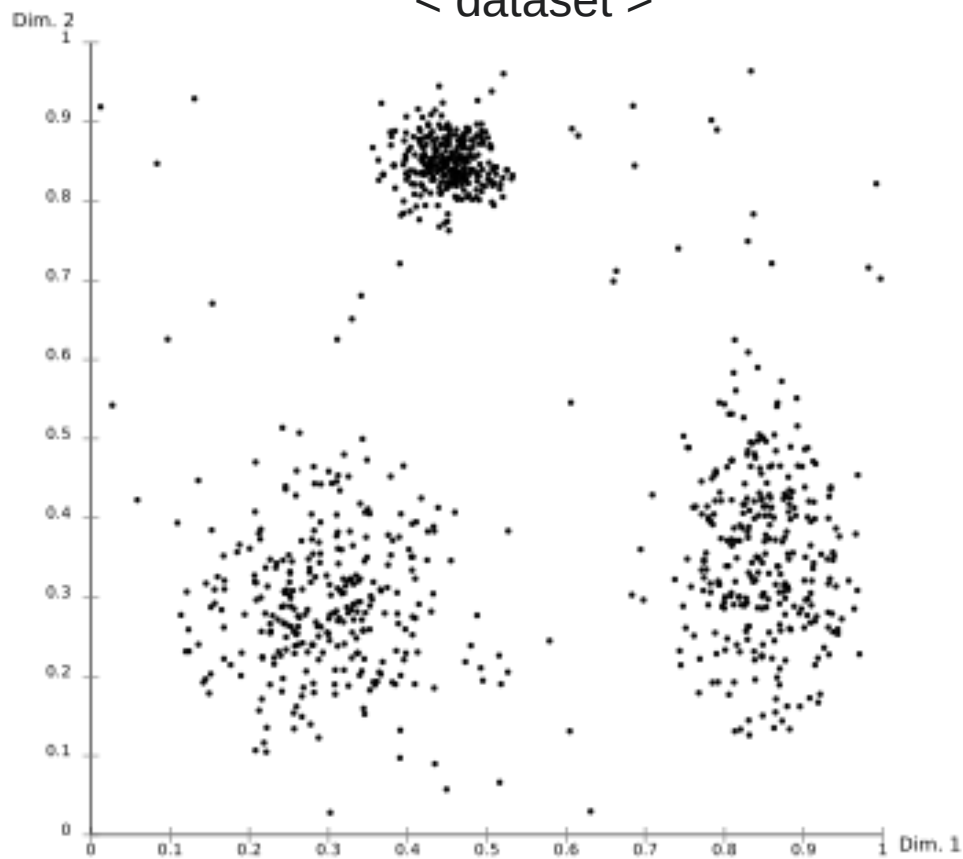




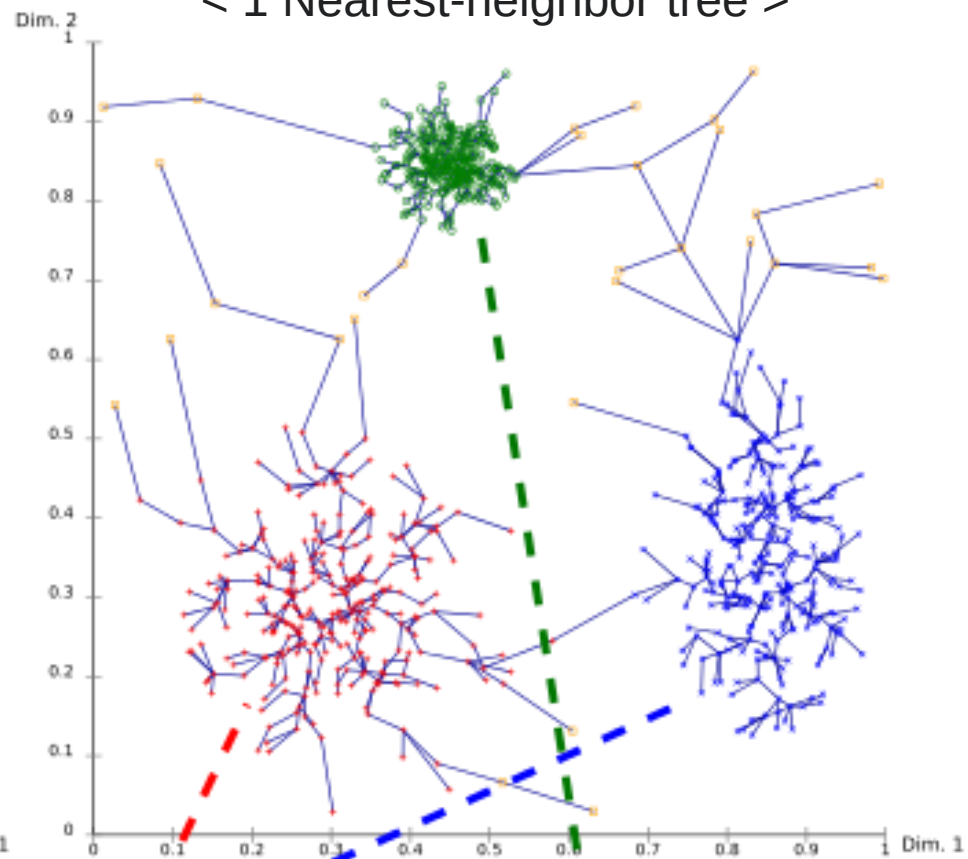
OPTICS: The Reachability Plot



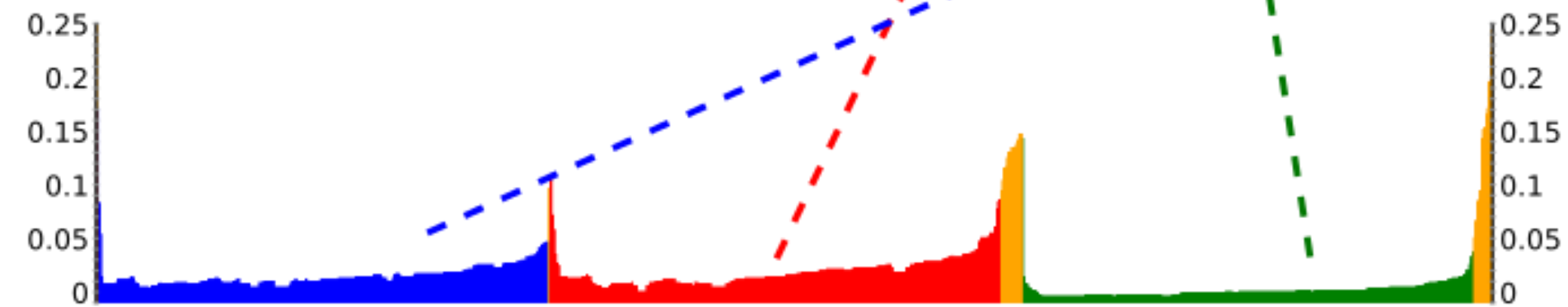
< dataset >



< 1 Nearest-neighbor tree >



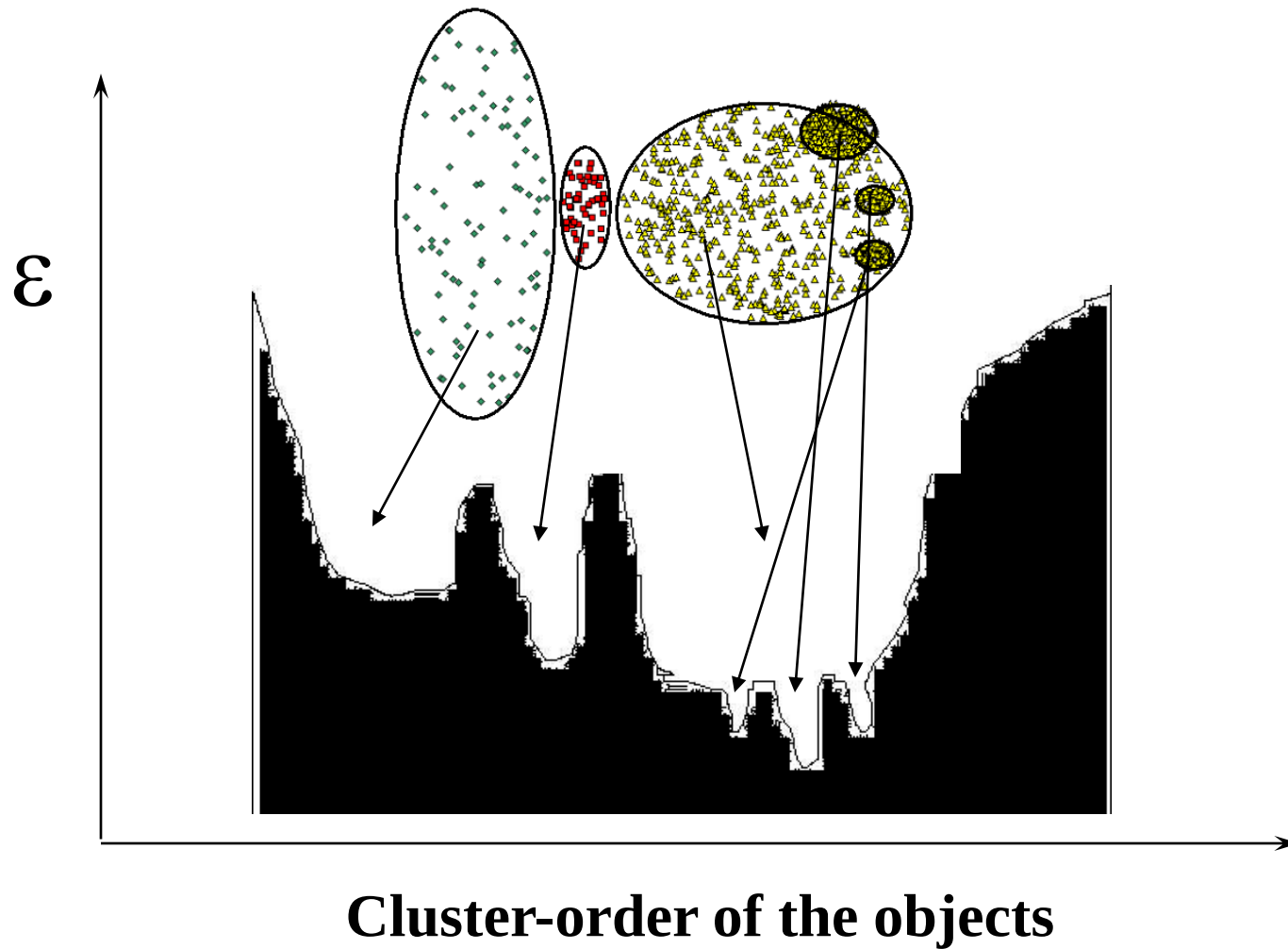
distance



order of objects

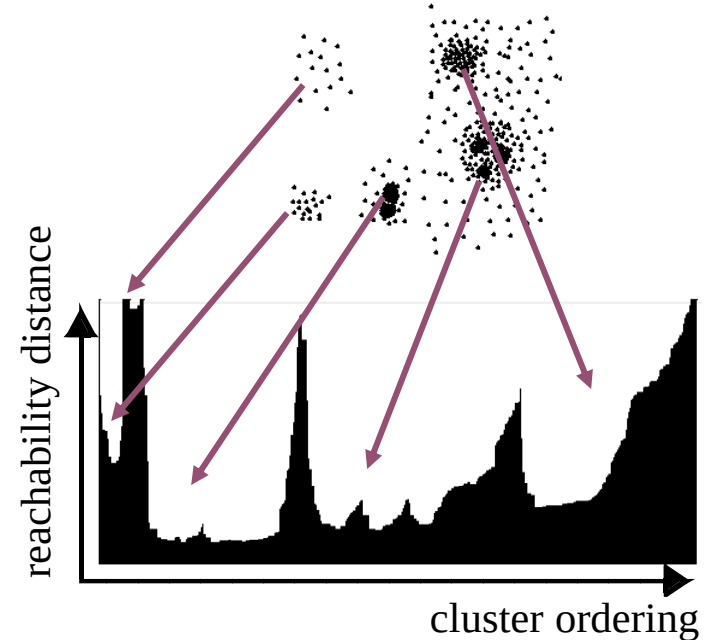
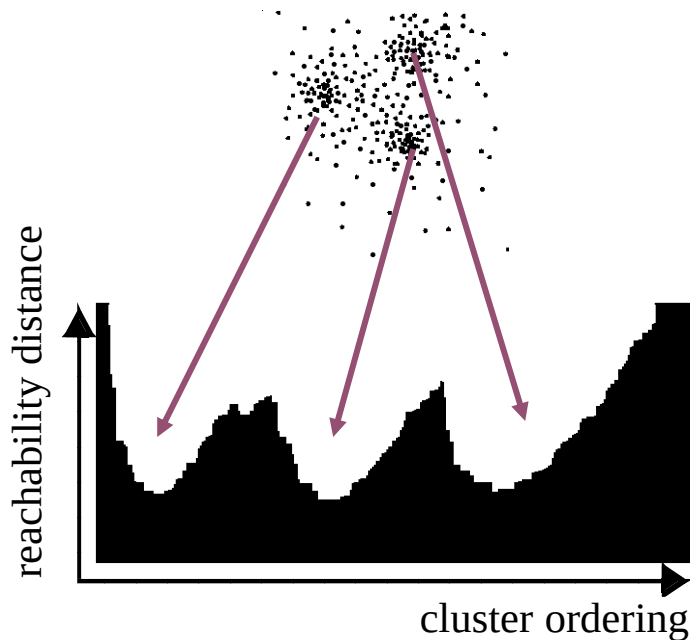


OPTICS: The Reachability Plot



OPTICS: The Reachability Plot

- ❑ Represents the density-based clustering structure
- ❑ Easy to analyze
- ❑ Independent of the dimension of the data



DBSCAN VS OPTICS

	DBSCAN	OPTICS
Density	Boolean value (high/low)	Numerical value (core distance)
Density- connected	Boolean value (yes/no)	Numerical value (reachability distance)
Searching strategy	random	greedy

Thank You